

11강. 서비스 메시와 API G/W

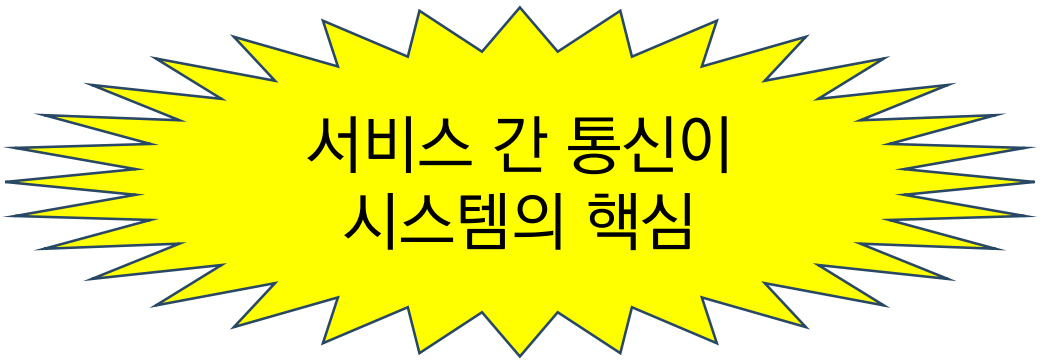


이영곤



문제 상황

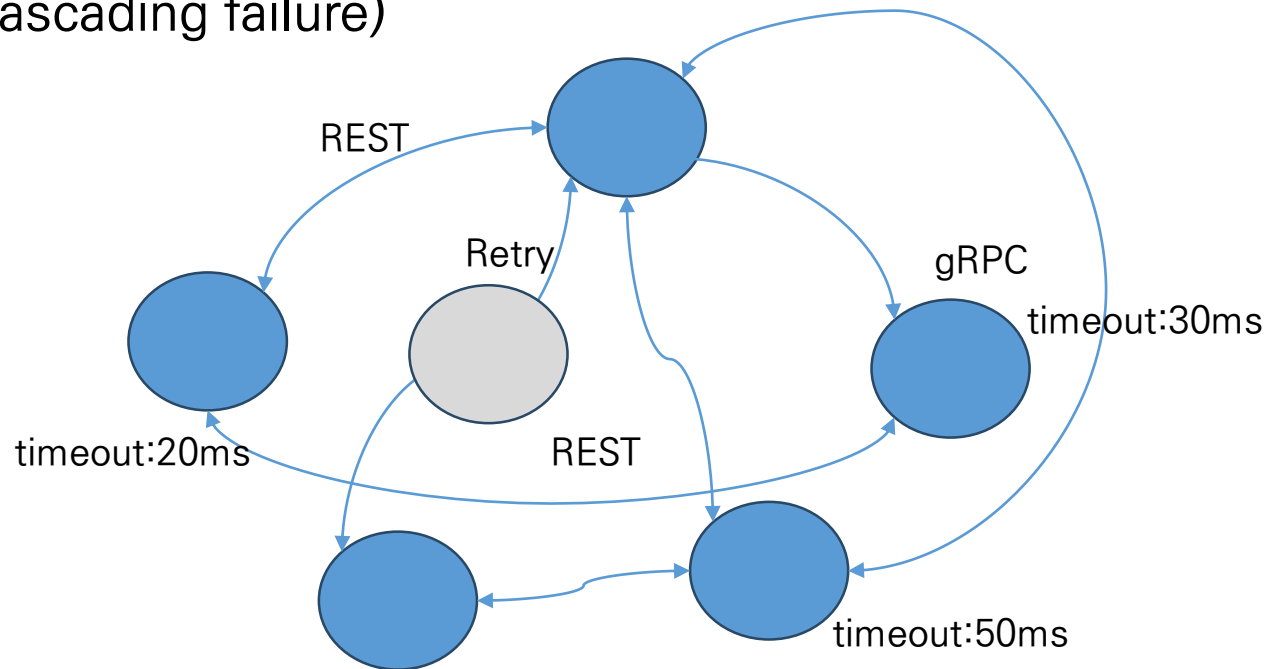
- ✓ 마이크로서비스 증가 → 통신 복잡도 증가
- ✓ 서비스 간 호출 경로 불명확
- ✓ 장애 발생 시 원인 추적 어려움



서비스 간 통신이
시스템의 핵심

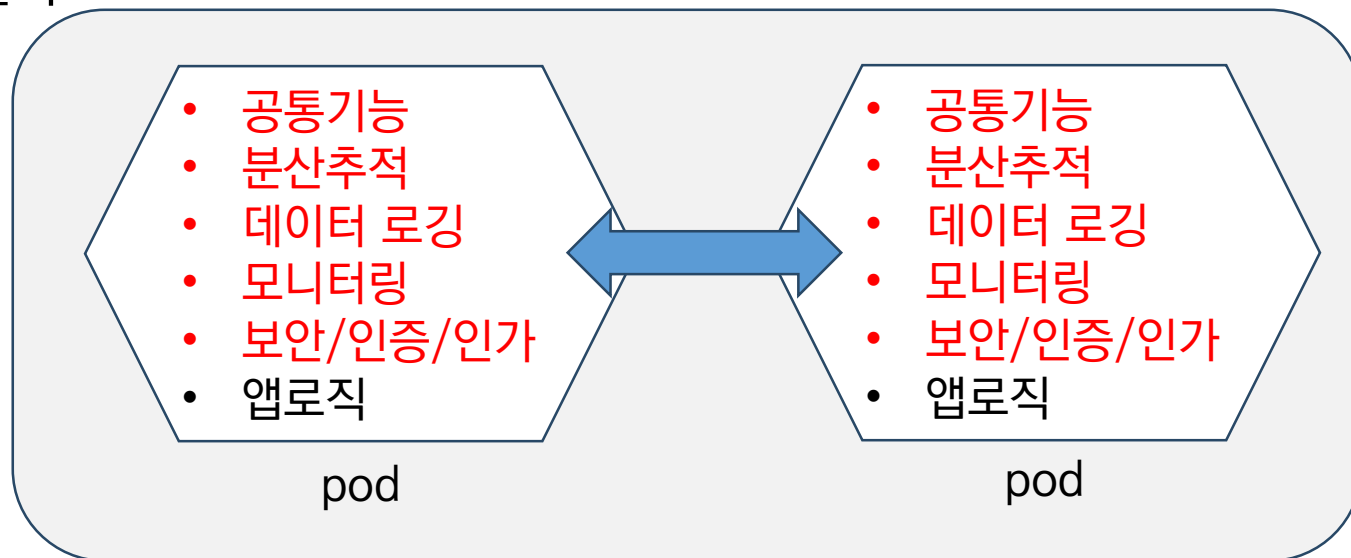
MSA 환경의 통신 문제

- ✓ 서비스 간 REST 호출 난립
- ✓ 타임아웃/재시도 로직 각 서비스에 중복 구현
- ✓ 보안 (TLS) 직접 구현 필요
- ✓ 장애 전파 (cascading failure)



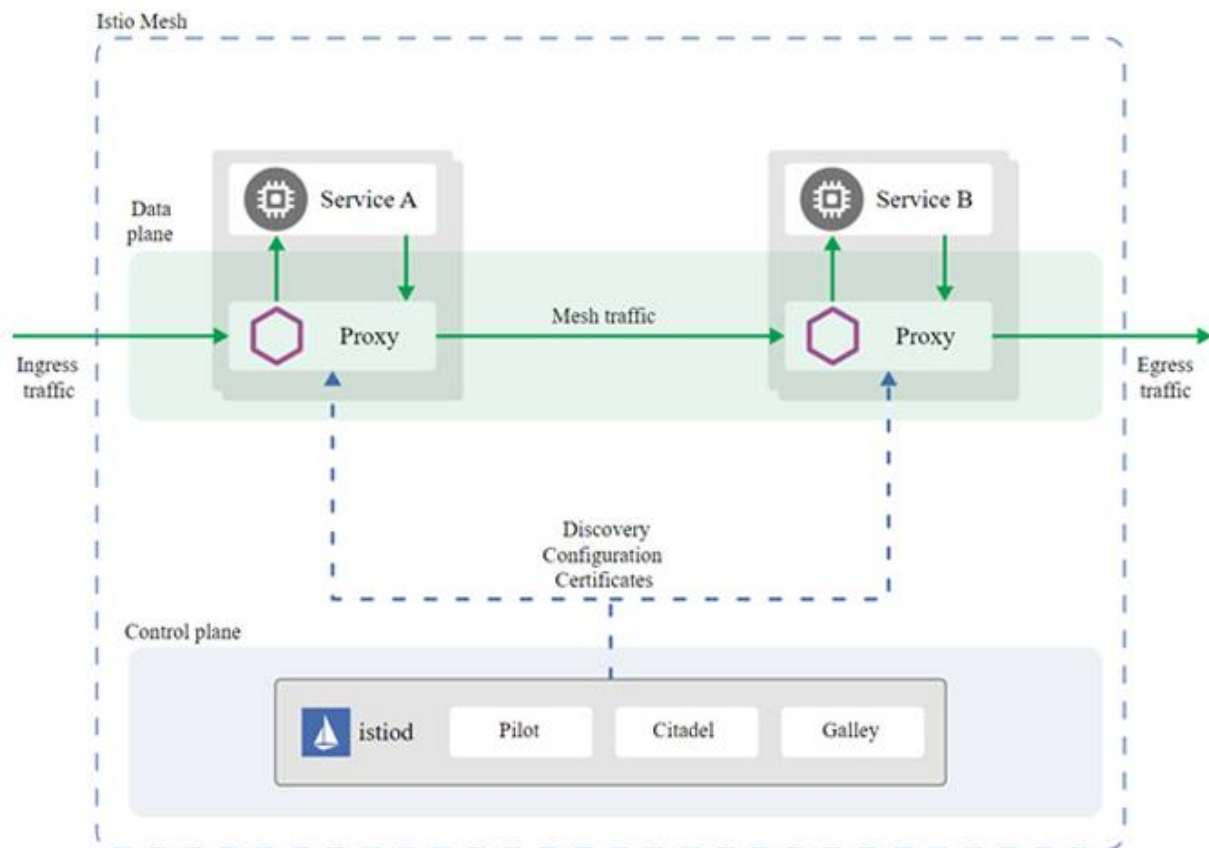
해결 방안

- ✓ retry, timeout, circuit breaker 직접 구현
- ✓ 문제
 - 서비스마다 구현 중복
 - 유지보수 어려움
 - 표준화 불가



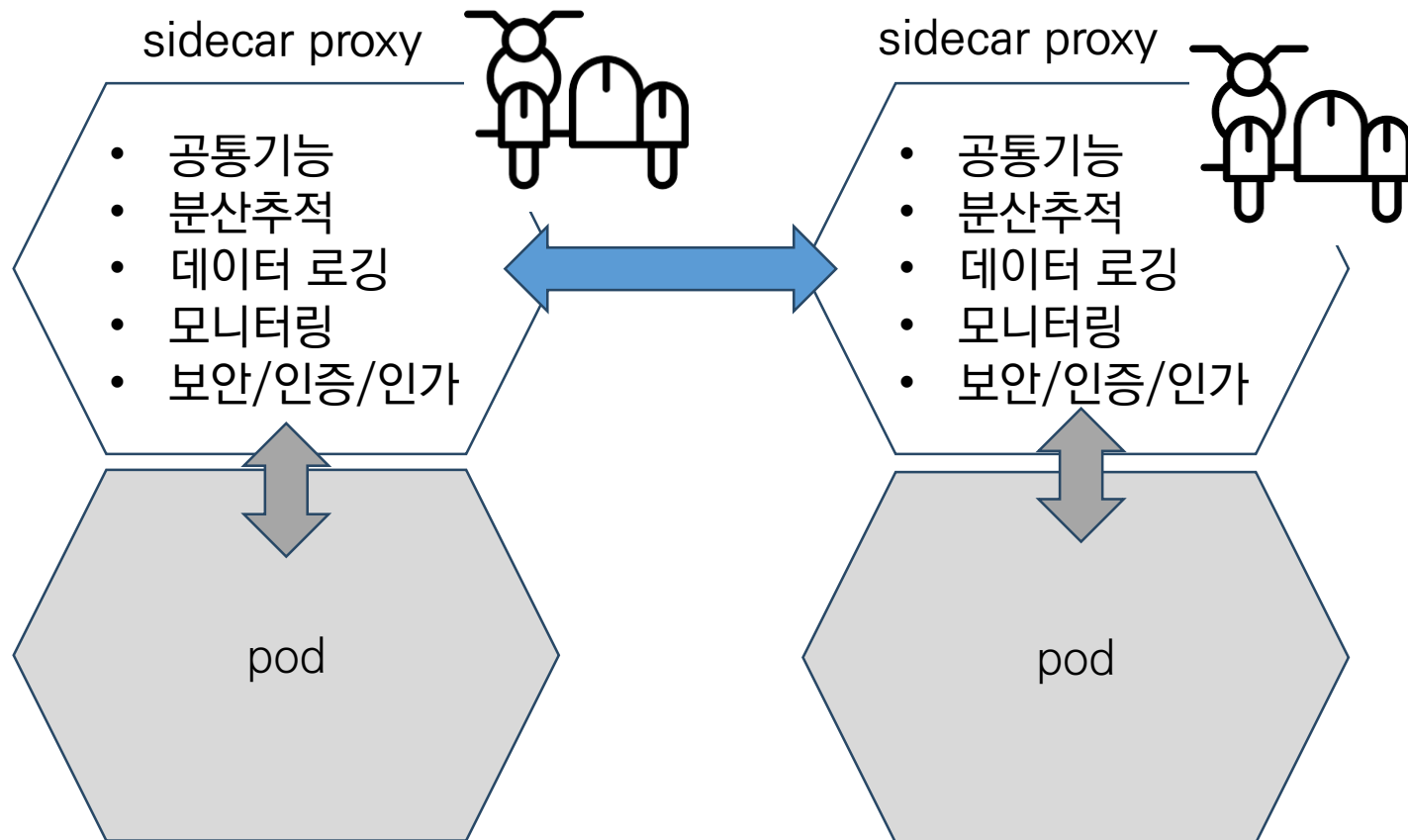
해결 방안

- ✓ 서비스 간 통신을 인프라 레벨에서 제어하는 구조
- ✓ 특징
 - 애플리케이션 코드 수정 없이 동작
 - 통신을 proxy가 대신 처리



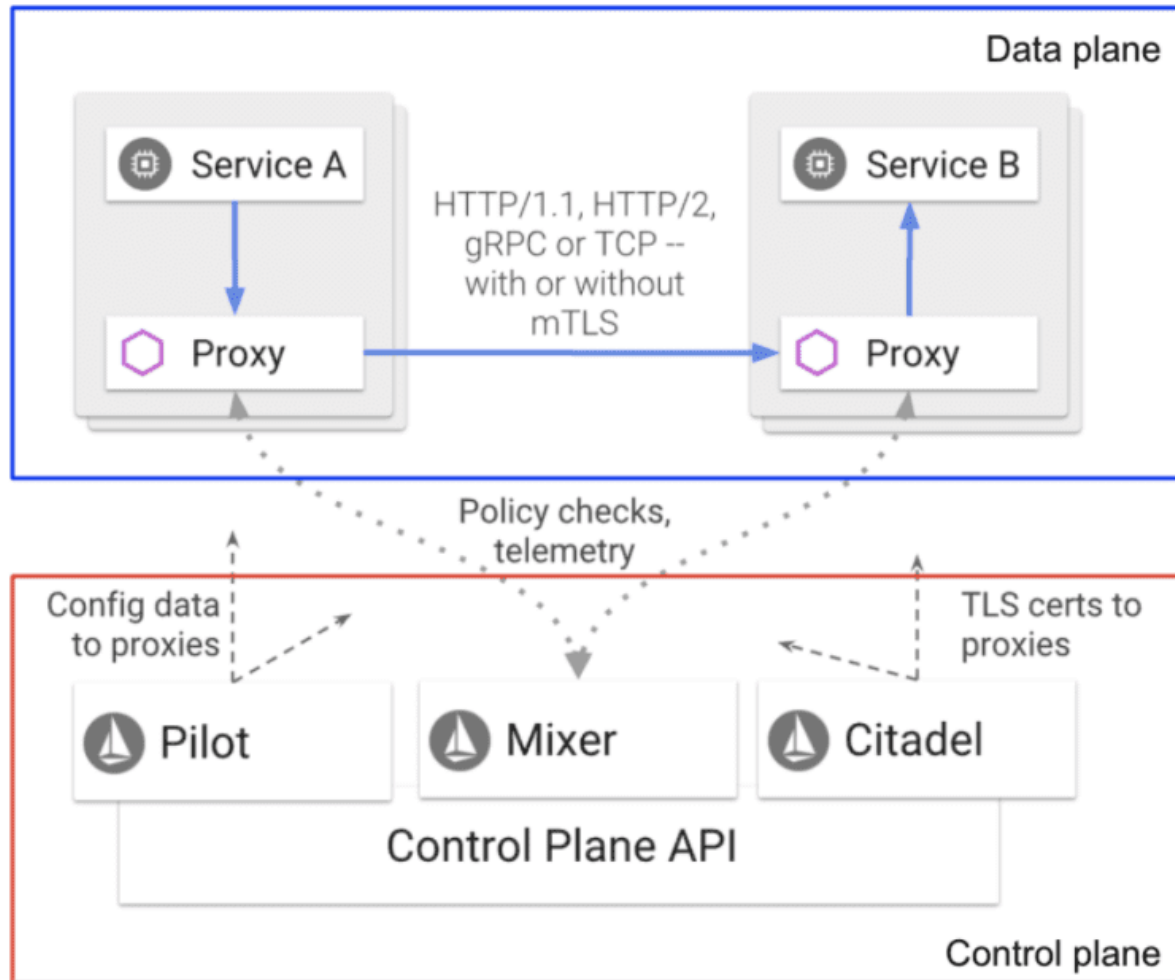
서비스 메시 기능

- ✓ 트래픽 제어
- ✓ 분산 추적
- ✓ 데이터 로깅
- ✓ 모니터링
- ✓ 보안/인증/인가



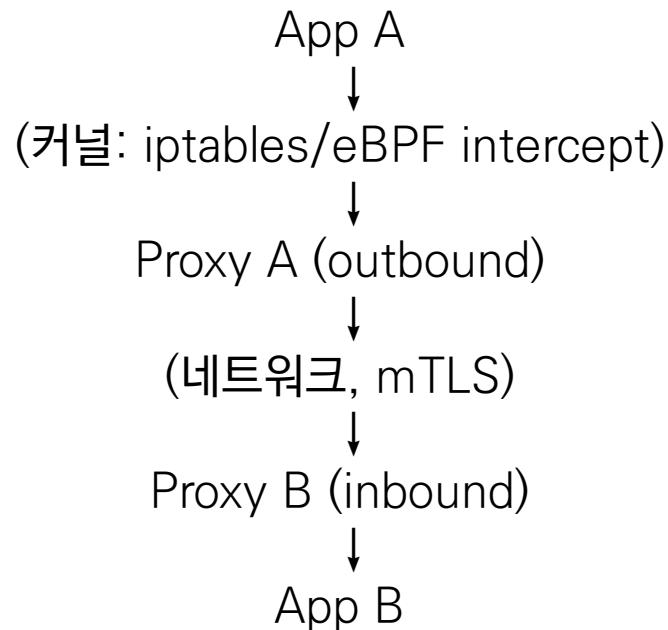
주요 기능

- ✓ Traffic Control
 - Load balancing
 - Retry / Timeout
- ✓ Observability
 - Latency
 - Success rate
 - Call graph
- ✓ Security
 - mTLS 자동 적용



프록시 통신 구조

- ① A 서비스는 일반 서비스명 호출
- ② iptables가 트래픽을 A의 Envoy로 강제 리다이렉트
- ③ A Envoy가 Istiod로부터 B 엔드포인트 정보 공유 받음
- ④ istiod가 xDS API를 통해 제공:
 - B 서비스 Pod 목록
 - 각 Pod의 Envoy 주소
 - 로드밸런싱 정책
 - mTLS 인증서
 - 라우팅 규칙
- ⑤ A Envoy가 적절한 B Envoy 선택
 - 예: Round Robin, Least Request, Canary Routing, Version Routing



단점

- ✓ 리소스 오버헤드 (가장 현실적인 문제)
 - 모든 Pod에 sidecar proxy 추가
 - CPU / Memory 증가 (대략 10~30%)
 - 노드 수 증가 필요
- ✓ 네트워크 지연 증가
 - 모든 요청이 proxy를 거침
 - App → Proxy → Network → Proxy → App
 - latency 증가 (수 ms ~ 수십 ms)

단점

- ✓ 디버깅 난이도 상승
 - 문제 발생 시: 문제 위치?– 애플리케이션?– 네트워크?– proxy?– mesh 정책?
 - 원인 분리 어려움
- ✓ 아키텍처 복잡성 증가
 - 추가되는 요소: Control Plane, Data Plane (proxy), CRD (ServiceProfile 등), 정책 (retry, timeout 등)
- ✓ 학습 비용
 - mTLS 개념
 - 트래픽 정책
 - observability
 - sidecar 구조

도입시 고려사항

- ✓ 정말 필요한가? (가장 중요)
 - 서비스 수 많음 (10개 이상)
 - 서비스 간 호출 복잡
 - 장애 분석 어려움
 - observability 필요
- ✓ retry / timeout 정책 설계
 - 너무 aggressive → 장애 증폭
 - 너무 conservative → 장애 발견 지연
- ✓ mTLS 영향 고려
 - 내부 통신 암호화 → CPU 사용 증가
 - 인증 체계 변경

도입시 고려사항

- ✓ observability 활용 준비: 메시 도입 이유의 70%는 관측성 때문
 - p95 latency
 - success rate
 - call graph
- ✓ 운영 자동화 필요
 - 모니터링 (Prometheus, Grafana)
 - 알람
 - 로그 수집

설치 순서

- ✓ WSL 이나 Git bash 설치 (git bash가 가장 간단)
- ✓ Gateway API 설치
- ✓ Linkerd CRD 설치
- ✓ Linkerd control plane 설치 시 `--set proxyInit.runAsRoot=true` 추가
- ✓ linkerd check로 검증

Linkerd 설치 스크립트 (도커 데스크탑 기준)

- ✓ `linkerd uninstall | kubectl delete -f -`
 - ✓ `kubectl delete ns linkerd`
 - ✓ `curl -sL https://run.linkerd.io/install | sh`
 - ✓ `kubectl apply --server-side -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/standard-install.yaml`
 - ✓ `linkerd install --crds | kubectl apply -f -`
 - ✓ `linkerd install --set proxyInit.runAsRoot=true | kubectl apply -f -`
 - ✓ `linkerd check`
 - ✓ `kubectl get pods -n linkerd`
 - ✓ `kubectl get configmap -n linkerd`
- 처음 설치시 안해도 됨
- Linkerd CLI 설치 후 path 반영

Linkerd 주입

- ✓ `kubectl annotate ns default linkerd.io/inject=enabled`
- ✓ `kubectl rollout restart deploy`

Feign 통신 대상 확인

- ✓ openfeignclient
- ✓ resttemplateserver

sidecar proxy가 떴어서
pod가 2개로 보임

```
pod/mysql-100d11f4dc-xngf1      2/2      Running      0          3m12s
pod/openfeignclient-68769d9975-ss89z  2/2      Running      0          3m12s
pod/resttemplateserver-569654469f-lnxnt 2/2      Running      1 (2m17s ago) 3m12s
```

sidecar 주입 대상

- ✓ REST API (pub/sub, producer/consumer etc.)
- ✓ gRPC
- ✓ 내부 TCP 호출
- ✓ mTLS
- ✓ 트래픽 제어

sidecar 주입시 신중히 고려 대상

- ✓ Kafka, RabbitMQ 같은 이벤트 브로커/메시지큐
- ✓ 자체 보안기능, 클러스터링, 포트 구조 등 가지고 있기 때문

Linkerd 주입 후 확인

- ✓ Pod 안에 container가 2개 생김
 - `kubectl get pods`
 - `kubectl describe pod <pod명>`
- ✓ app container + linkerd-proxy
- ✓ 모든 서비스 간 호출이 proxy 경유 (확인)
 - `linkerd viz install | kubectl apply -f -`
 - `linkerd viz check`
 - `linkerd viz stat deploy -A` (통계 정보)
 - `linkerd viz edges deploy -A` (메시징 정보)
 - `linkerd viz dashboard` (대시보드 화면)

Linkerd Biz 대시보드: 네임스페이스

localhost:50750/namespaces

LINKERD

Namespaces

CLUSTER

- Namespaces
- Control Plane

LINKERD-VIZ

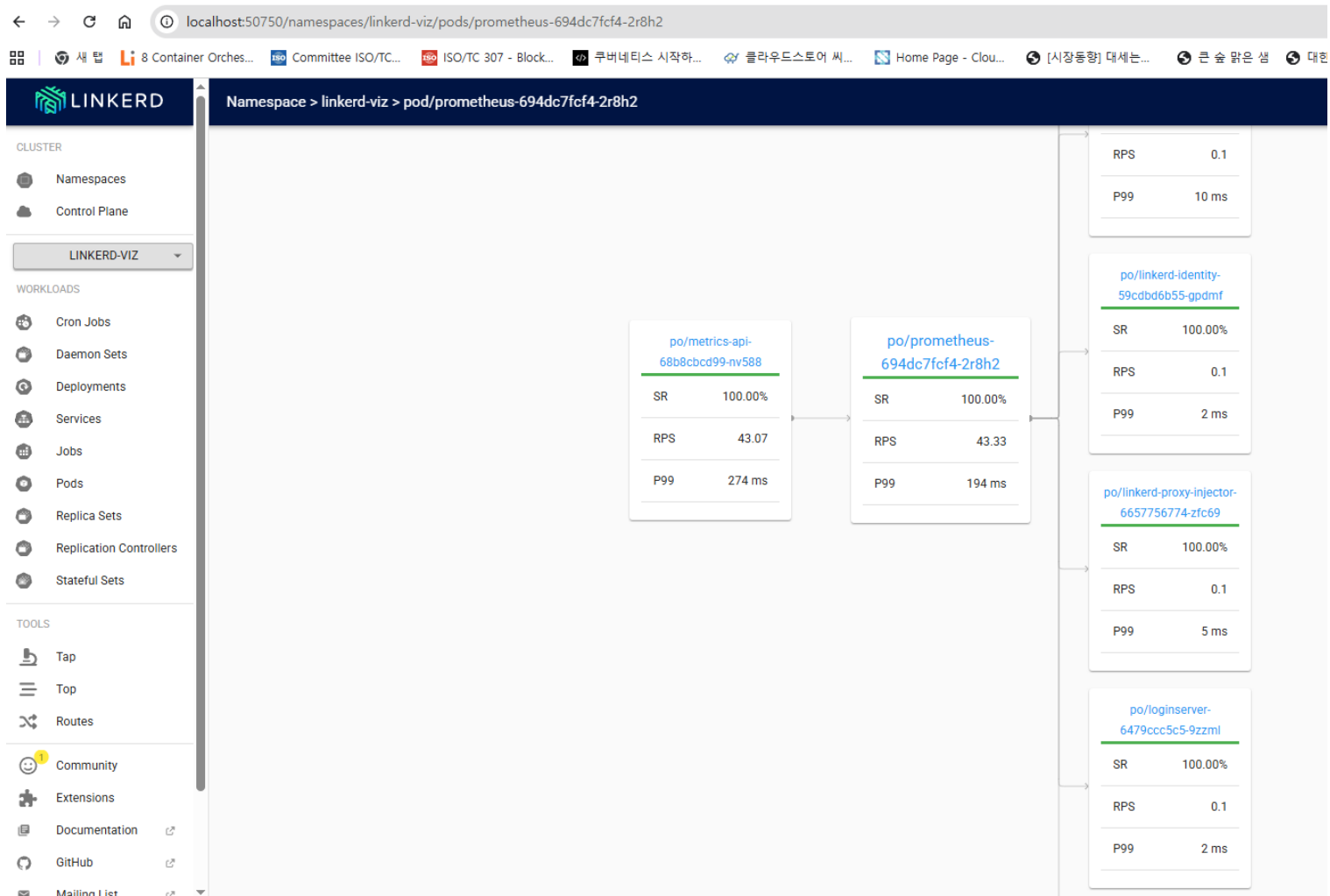
WORKLOADS

- Cron Jobs
- Daemon Sets
- Deployments
- Services
- Jobs
- Pods
- Replica Sets
- Replication Controllers

HTTP Metrics

Namespace ↑	↑ Meshed	↑ Success Rate	↑ RPS	↑ P50 Latency	↑ P
book-dev	0/2	---	---	---	
default ↺	6/6	100.00% ●	1.98	1 ms	
ingress-nginx	0/1	---	---	---	
kube-node-lease	0/0	---	---	---	
kube-public	0/0	---	---	---	
kube-system	0/9	---	---	---	
linkerd	3/3	100.00% ●	2.08	1 ms	
linkerd-viz	5/5	100.00% ●	3.58	2 ms	

Linkerd 메트릭 정보 확인(openfeignclient와 resttemplateserver)



용어 의미

- ✓ SR (Success Rate)
 - 성공률
- ✓ RPS (Requests Per Second)
 - 초당 요청 수
 - 예: prometheus: 45.1 req/sec, mysql: 0.1 req/sec
- ✓ P99
 - 99% 요청이 이 시간 이하로 처리됨
 - mysql: 3ms, prometheus: 99ms

Linkerd yaml 설명

```
apiVersion: linkerd.io/v1alpha2
```

```
kind: ServiceProfile
```

```
metadata:
```

```
  name: payment.default.svc.cluster.local
```

```
  namespace: default
```

```
spec:
```

```
  routes:
```

```
    - name: GET /payment
```

```
      condition:
```

```
        method: GET
```

```
        pathRegex: /payment
```

```
        isRetryable: true
```

```
        timeout: 1s
```

```
      retryBudget:
```

```
        retryRatio: 0.2
```

```
        minRetriesPerSecond: 10
```

```
        ttl: 10s
```

Retry 비율 제한
10개 요청중 2개까지
Retry 허용

초당 최소 Retry 횟수

최근 몇초동안 요청을
기준으로 할 것인가?

Linkerd yaml

✓ 타임아웃

- timeout: 1s $\Rightarrow$ 1초 초과 시 실패 처리
- 응답 지연 방지, 시스템 전체 병목 완화
- 요청 강제 실패 처리, upstream 연결 종료, 클라이언트에 에러 반환, retry 조건이면 재시도

✓ retryRatio: 0.2 $\Rightarrow$ 원 요청 대비 최대 20% 재시도 허용

- 예: 초당 100 요청 $\rightarrow$ 최대 20 재시도

✓ minRetriesPerSecond: 10 $\Rightarrow$ 초당 최소 10번 재시도 하라고 지정

- retry 횟수를 보장하기 위한 장치
- 예를 들어 1로 할 경우, retryRatio가 0.2면 거의 retry가 발생하지 않음

✓ TTL(Time To Live) : 10 $\Rightarrow$ 최근 10초간 요청량 기준

메시 삭제 (git bash에서 실행)

- ✓ `linkerd viz uninstall | kubectl delete -f -`
- ✓ `linkerd uninstall | kubectl delete -f -`
- ✓ `kubectl delete crd -I linkerd.io/control-plane-ns=linkerd`
- ✓ `kubectl delete namespace linkerd`
- ✓ `kubectl delete namespace linkerd-viz`
- ✓ `kubectl rollout restart deployment`

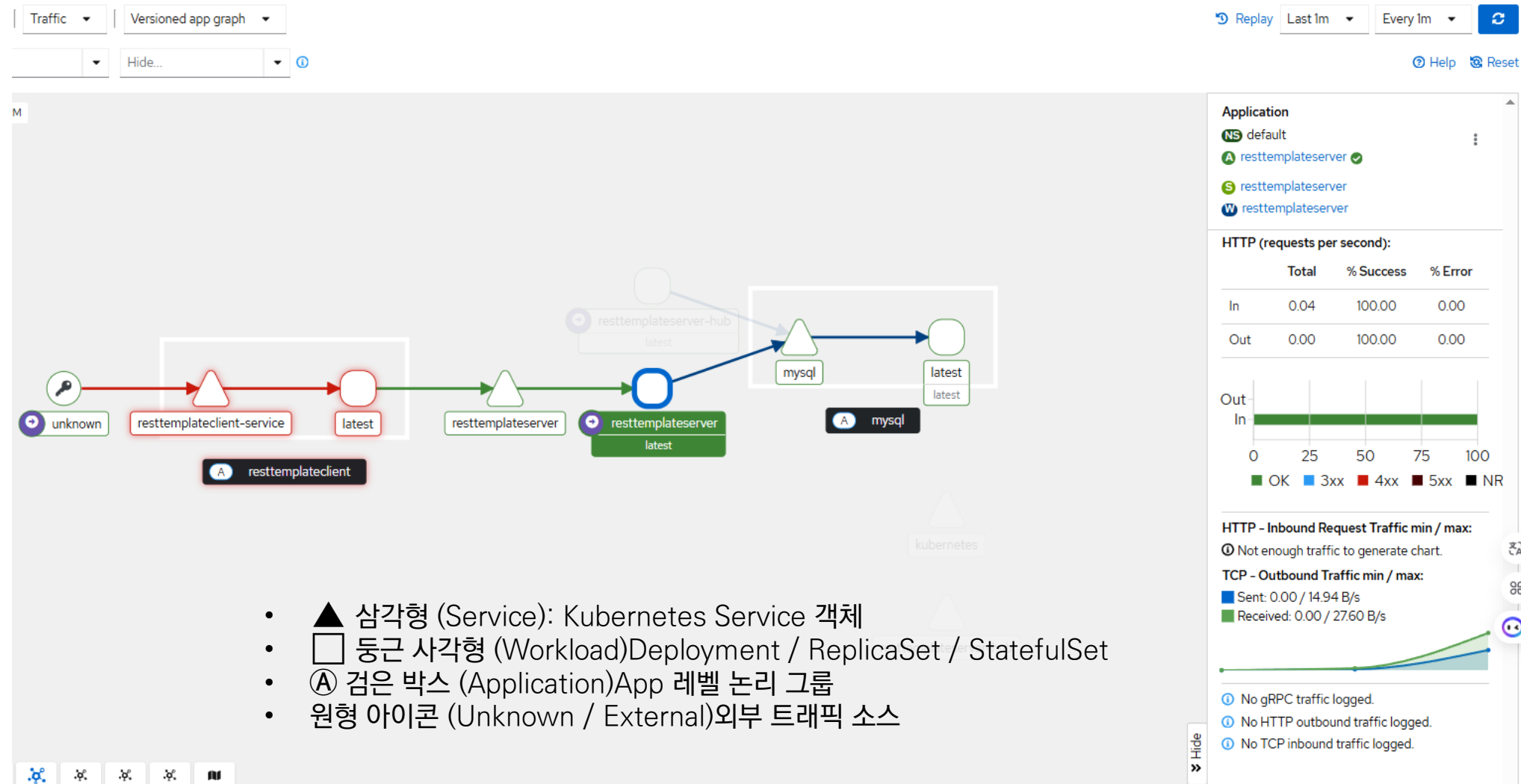
- ✓ `Invoke-WebRequest -Uri https://github.com/istio/istio/releases/download/1.27.3/istio-1.27.3-win.zip -OutFile istio.zip`
- ✓ `Expand-Archive istio.zip -DestinationPath .`
- ✓ `cd istio-1.27.3`
- ✓ `istioctl install --set profile=demo -y`
- ✓ `kubectl get pods -n istio-system`
- ✓ `kubectl label namespace default istio-injection=enabled`
- ✓ `kubectl apply -f samples/addons`
- ✓ `kubectl get pods -n istio-system`
- ✓ `istioctl dashboard kiali`
- ✓ `istioctl dashboard Grafana`
- ✓ `istioctl dashboard prometheus`
- ✓ `kubectl rollout restart deployment`

* powershell 에서 실행

Kiali

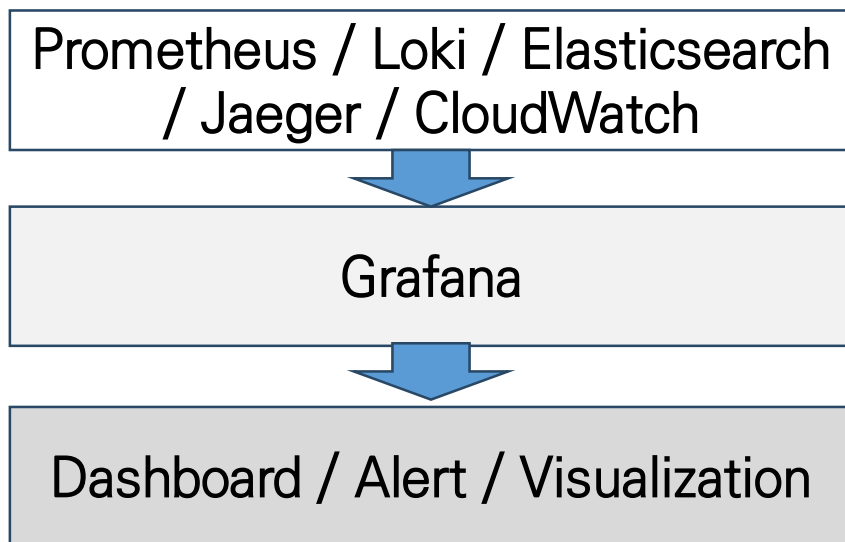
- ✓ 서비스 메시 환경을 시각적으로 관리·분석하는 관측성(Observability) 콘솔
 - 서비스 간 호출 관계를 실시간 지도처럼 보여주는 관리 대시보드
- ✓ 서비스 메시 토폴로지 시각화
- ✓ 트래픽 흐름 확인
 - 요청량(Request Rate), 성공률, 에러율, 응답시간(Latency)
- ✓ 사이드카 상태 확인
 - Sidecar 존재 여부, Missing Sidecar 탐지, mTLS 적용 여부
- ✓ 설정 검증
 - VirtualService, DestinationRule, Gateway, AuthorizationPolicy
- ✓ 보안 상태
 - mTLS 활성화 여부, 암호화 여부, 서비스 간 보안 정책

Kiali



Grafana

- ✓ 시스템·애플리케이션·인프라의 메트릭, 로그, 트레이스를 시각화하는 통합 모니터링 대시보드 플랫폼
 - 운영 데이터(성능/상태)를 실시간 차트와 대시보드로 보여주는 분석 도구
- ✓ 운영 상태 가시화:
 - CPU, Memory, Pod 상태, API 응답속도, Error Rate, Kafka 처리량, DB 부하, Kubernetes 상태, Istio 서비스 메트릭



Grafana 주요 기능

- ✓ 대시보드
 - 실시간 차트, 히트맵, 게이지, 테이블, 알람 패널
- ✓ Alerting
 - CPU 90% 이상, Error 급증, Pod down, Latency 증가
- ✓ 로그 분석
 - Loki 연동 시: 에러 로그 검색
- ✓ Kubernetes 모니터링
 - Pod 상태, Namespace별 자원, Deployment 성능
- ✓ Istio와 함께 사용 시
 - Request Rate, Success Rate, P95/P99 Latency, Service Traffic, mTLS 상태 일부

Grafana 주요 기능(로그 확인하기)

Kubernetes + Grafana + Loki + Promtail 로그 구축

- ✓ Pod Logs
 - Promtail 로그 수집
 - Loki 로그 저장
 - Grafana 조회
- ✓ `kubectl get svc -n istio-system | findstr loki` (로키 설치여부 확인)
- ✓ `kubectl port-forward -n istio-system svc/loki 3101:3100`
- ✓ Invoke-WebRequest <http://localhost:3101/ready> (로키상태 확인)

Grafana 주요 기능(로그 확인하기: promtail 설치)

- ✓ `winget install Helm.Helm` (powershell) 설치후 창을 새로 띄움
- ✓ `helm repo add grafana https://grafana.github.io/helm-charts`
- ✓ `helm repo update`
- ✓ `kubectl create namespace monitoring`
- ✓ `helm install promtail grafana/promtail --namespace monitoring --set config.clients[0].url=http://loki.monitoring.svc.cluster.local:3100/loki/api/v1/push`
- ✓ `kubectl get pods -n monitoring`
- ✓ `kubectl logs -n monitoring daemonset/promtail`

Grafana 주요 기능(promtail ⇒ Loki로 로그 적재 확인)

- ✓ Invoke-WebRequest <http://localhost:3101/loki/api/v1/labels>
 - {"status":"success","data":["namespace","pod","container","app",...]}
- ✓ Grafana에서 Loki Data Source 등록
 - <http://loki.istio-system.svc.cluster.local:3100>
- ✓ Grafana 로그 조회 (LogQL 활용)
 - {namespace="default"}
 - {app="resttemplateserver"}
 - {namespace="default"} |= "ERROR"

Grafana 로그 확인

Grafana Explore page for Loki logs. The interface shows the query editor with the query `{app="resttemplateclient"}` and the 'Builder' and 'Code' buttons. The 'Logs' tab is selected at the bottom. The main panel displays the 'Logs volume' chart and the 'Logs' tab with 4 lines displayed and total bytes processed: 1.74 kB. Common labels are: `app=resttemplateclient container=istio-proxy job=default/resttemplateclient`.

Grafana 주요 기능(프로메테우스를 데이터소스로 등록)

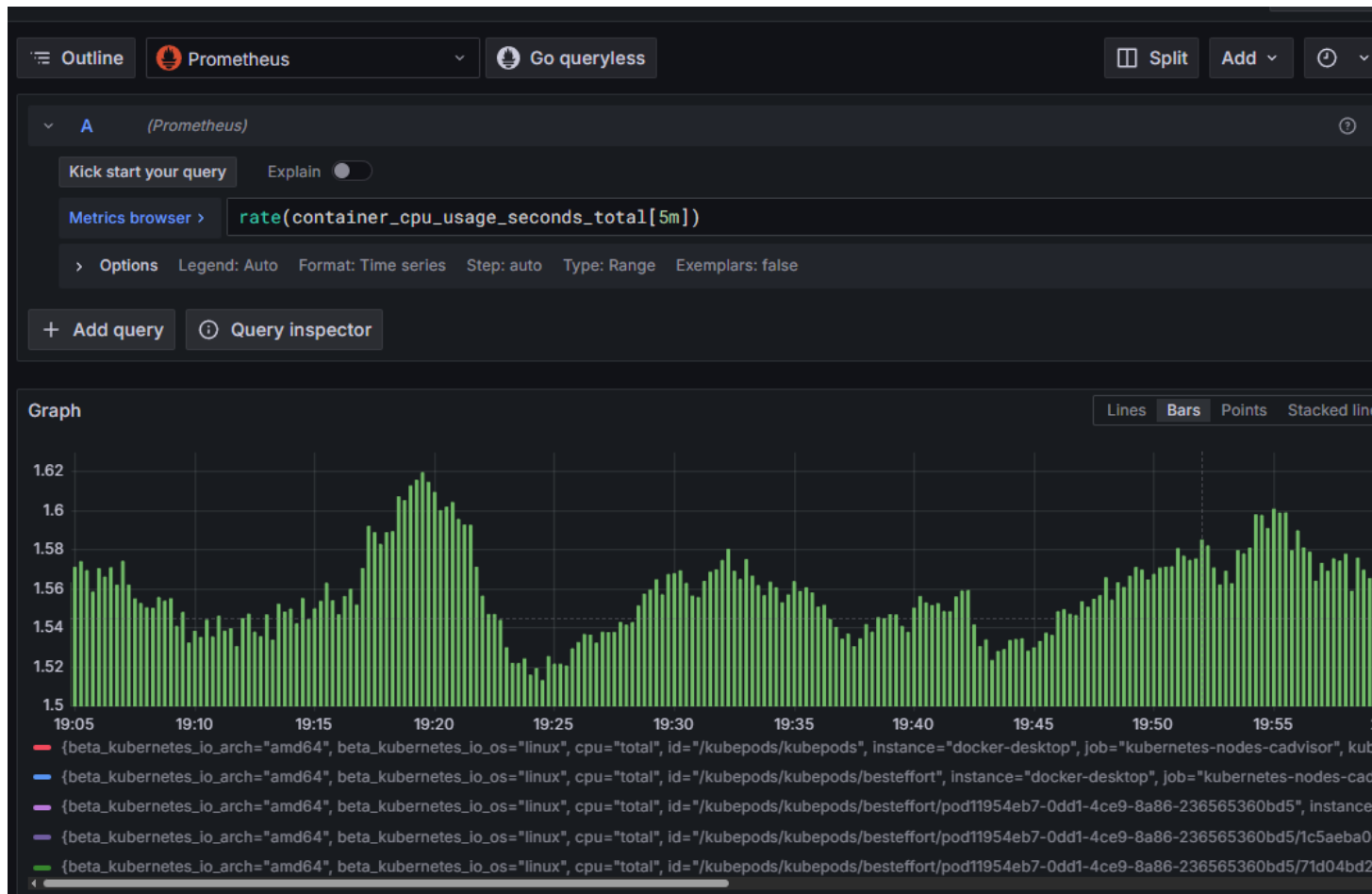
✓ 데이터 소스 등록

- <http://prometheus.istio-system.svc.cluster.local:9090>

항목	Prometheus UI	Grafana
PromQL 테스트	매우 좋음	좋음
대시보드	제한적	강력
시각화	기본	고급
로그 조회	불가	Loki 연동

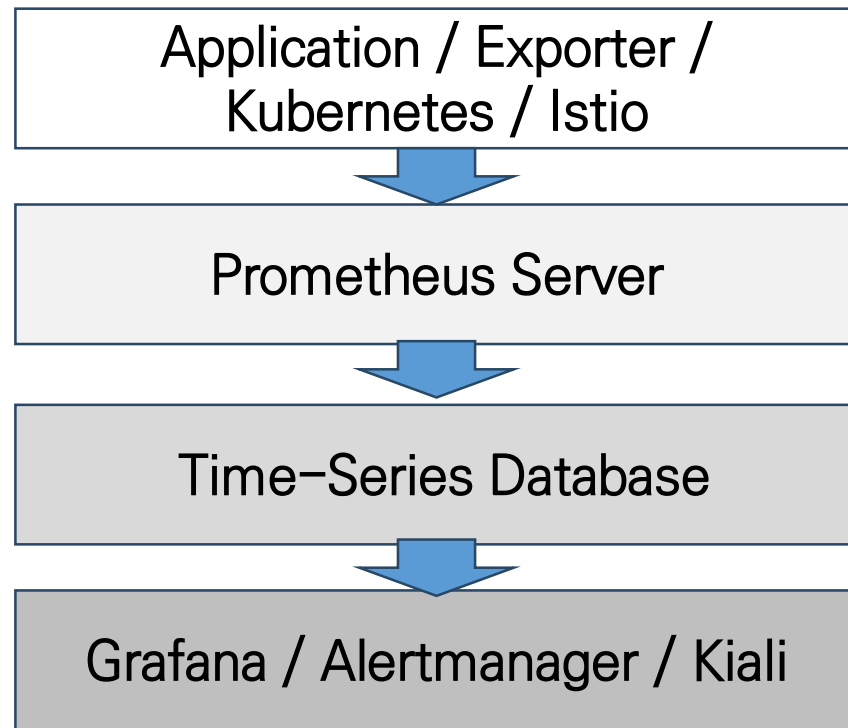
Grafana 주요 기능(프로메테우스를 데이터소스로 등록)

PromQL test: `rate(container_cpu_usage_seconds_total[5m])`



Prometheus 개념

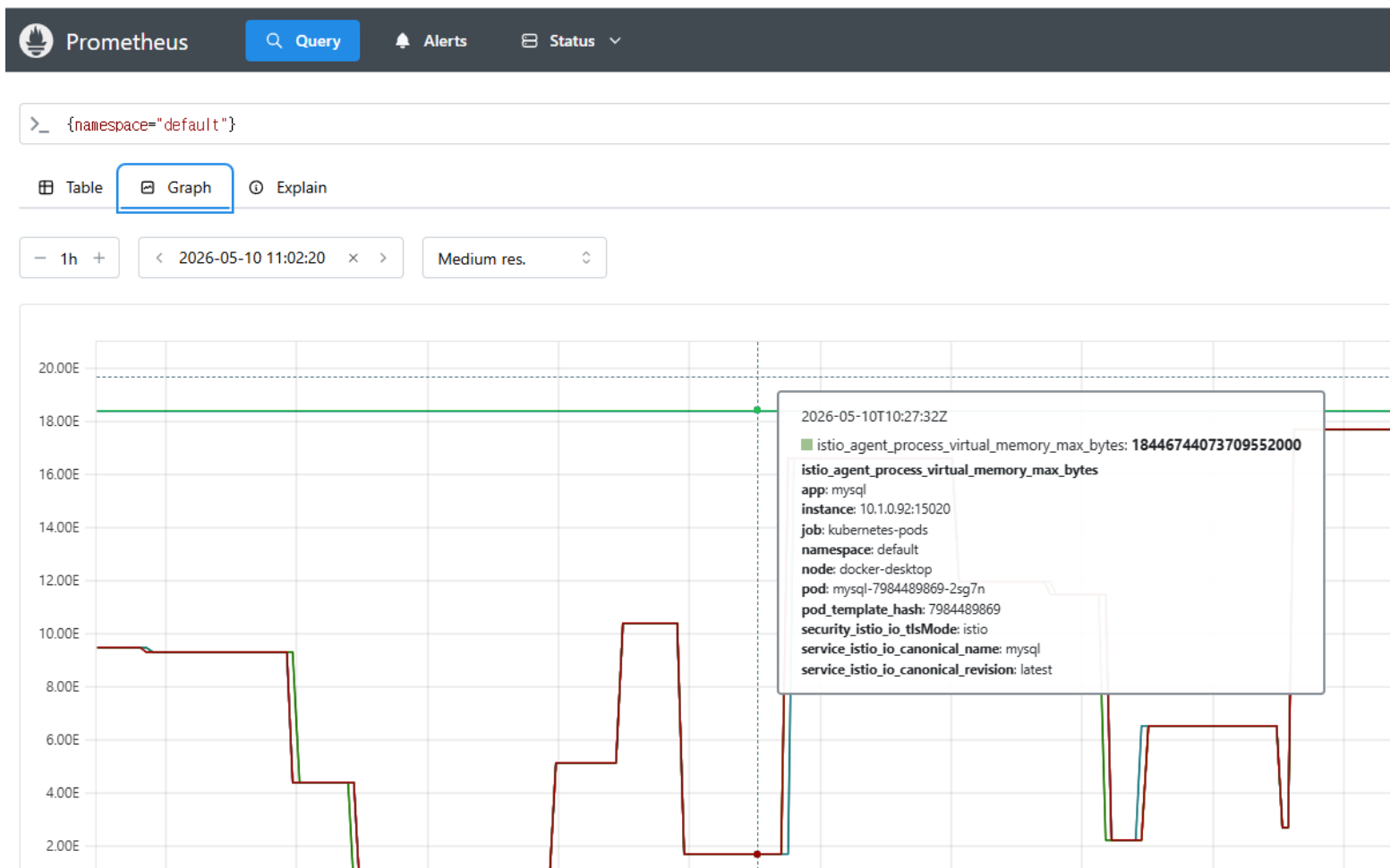
- ✓ 시스템과 애플리케이션의 메트릭(metrics)을 수집·저장·질의하는 오픈소스 모니터링 시스템
- ✓ 앱·쿠버네티스의 상태 수치를 주기적으로 끌어와 저장하는
 - 시계열 데이터베이스 + 모니터링 엔진
- ✓ 핵심 역할(주기적 수집)
 - CPU 사용률
 - 메모리 사용량
 - HTTP 요청 수
 - 에러율, 응답시간
 - Kafka lag, DB 연결 수



Prometheus 개념

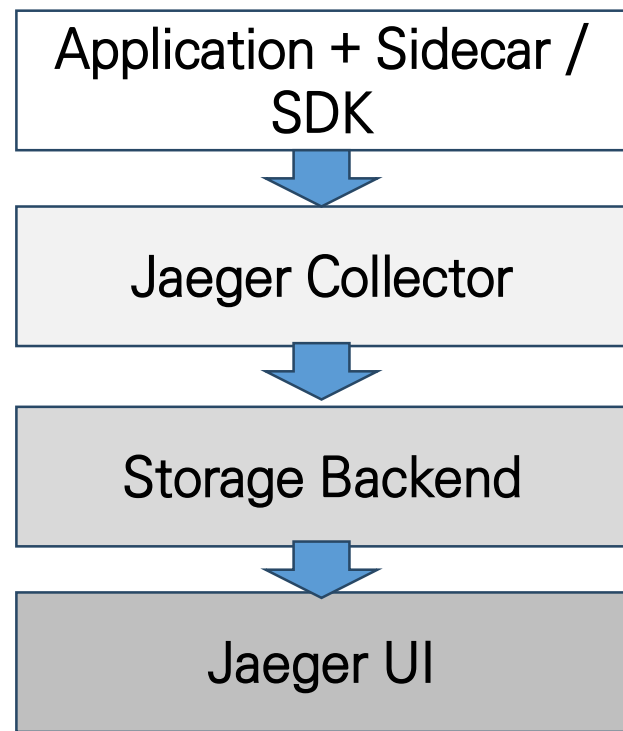
- ✓ Exporter(메트릭 제공기)
 - Node Exporter, MySQL Exporter, Kafka Exporter, cAdvisor, kube-state-metrics
- ✓ Pull 방식 사용
 - Prometheus가 직접 대상에 요청: GET /metrics
- ✓ Time-Series DB
 - 시간 기반 저장: `cpu_usage{pod="api-1"} 75`
- ✓ PromQL(전용 질의 언어): `rate(http_requests_total[5m])`
- ✓ 모니터링 가능: NodePod, Deployment, Namespace, HPA, Service Mesh, API latency
- ✓ Istio 환경: Request count, Error ratio, Service latency, mTLS telemetry
- ✓ Alertmanager 연계: CPU > 90%, Pod CrashLoop, 5xx 급증

Prometheus 실행



Jaeger 개념

- ✓ 마이크로서비스 간 요청 흐름을 추적(Distributed Tracing)하는 시스템
 - 하나의 사용자 요청이 여러 서비스들을 어떻게 거쳐 처리되는지 시간 흐름으로 추적하는 도구
 - Istio sidecar 를 통해 자동으로 trace 생성
 - 각 서비스 호출을 Trace / Span 단위로 기록
 - 어디서 느려졌는지? 어느 서비스가 오류인지?
 - 병목 구간은? 호출 순서는?
- ✓ 핵심 개념
 - Trace: 하나의 전체 요청 흐름
 - Span: 각 서비스 단계



Jaeger 기능

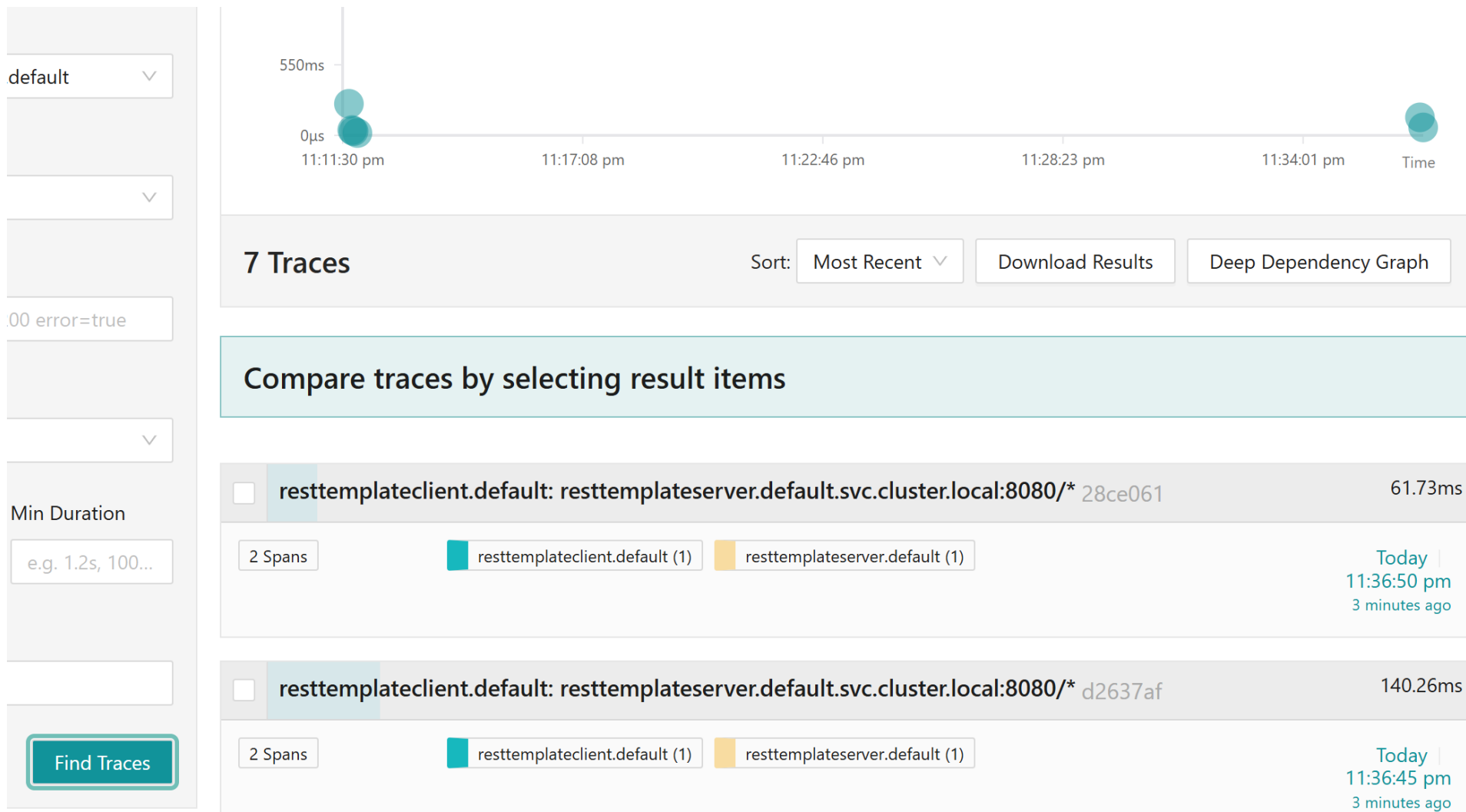
- ✓ 호출 경로 시각화
 - 서비스 순서, 병렬 호출, 지연 구간
- ✓ Latency 분석
 - 어느 서비스가 느린지, DB 쿼리 지연
- ✓ 오류 추적
 - 실패 서비스, timeout, retry
- ✓ Root Cause 분석
 - 예: payment-service DB timeout
- 예거 실행을 위해 트레이싱 정보를 이스티오에서 예거에게 넘겨주어야 함:
`istioctl install -f tracing.yaml -y`
`kubectl rollout restart deployment`

tracing.yaml

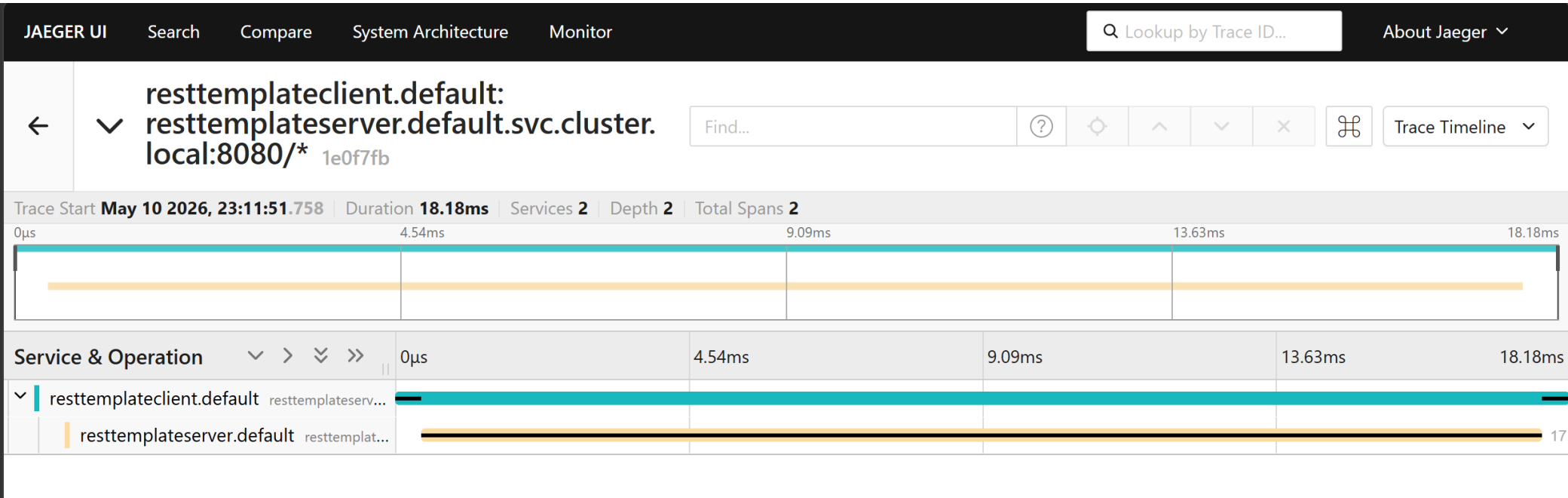
```
apiVersion: install.istio.io/v1alpha1
kind: IstioOperator
spec:
  meshConfig:
    enableTracing: true
    defaultConfig:
      tracing: {}
    extensionProviders:
      - name: jaeger
        opentelemetry:
          port: 4317
          service: jaeger-collector.istio-
system.svc.cluster.local
```

```
---
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: mesh-default
  namespace: istio-system
spec:
  tracing:
    - providers:
        - name: jaeger
          randomSamplingPercentage: 100
```


Jaeger 실행결과



Jaeger 실행결과



서비스 메시 G/W

✓ G/W 기본 정책 설정:

- 포트 개방
- 도메인
- HTTP/HTTPS
- TLS

```
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: resttemplate-gateway
  namespace: default
spec:
  selector:
    istio: ingressgateway
  servers:
    - port:
        number: 80
        name: http
        protocol: HTTP
      hosts:
        - "*"
```

Virtual Service

- ✓ Virtual Service 역할:
 - Path Routing
 - Version Routing
 - Header 기반 라우팅 결정
 - Retry / Timeout / Fault Injection

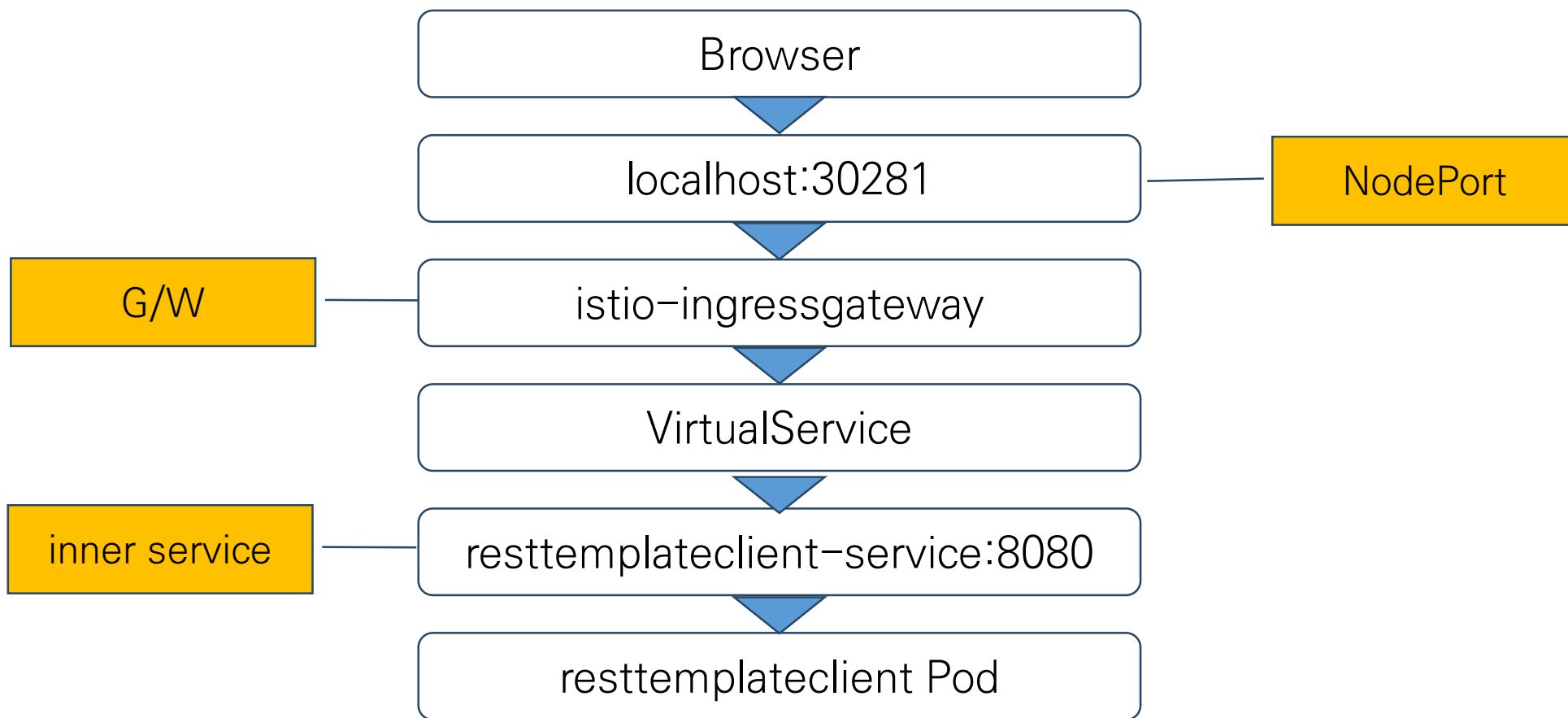


어디로 보낼지
결정

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: resttemplateclient-vs
  namespace: default
spec:
  hosts:
    - "*"
  gateways:
    - resttemplate-gateway
  http:
    - match:
        - uri:
            prefix: /
      route:
        - destination:
            host: resttemplateclient-
              service.default.svc.cluster.local
            port:
              number: 8080
```

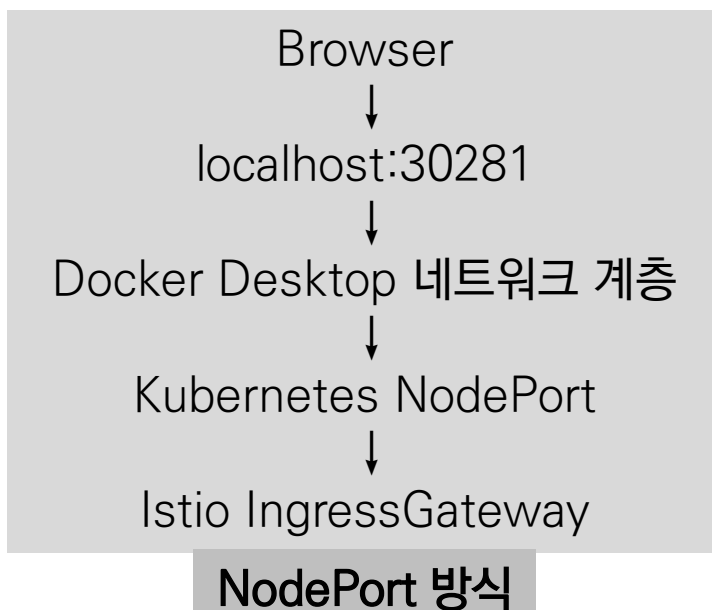
G/W + Virtual Service

- ✓ 웹브라우저로 접속: `http://localhost:30281`



G/W + Virtual Service

- ✓ <http://localhost:30281> → NodePort 방식인데 접속이 안됨
- ✓ port-forward로 진행: `kubectl port-forward -n istio-system svc/istio-ingressgateway 8088:80`
- ✓ <http://localhost:8088/index.html>, <http://localhost:8088/books>
- ✓ 도커데스크탑은 실제 노드가 아님. 따라서, 포트 충돌, localhost 리다이렉션 실패, 방화벽 차단, Docker networking bug, WSL2 bridge 문제 등 다양한 문제 발생 가능



NodePort/LoadBalancer를 써야 하는 경우: 실 운영시점

- ✓ 실운영하는 시점에서 사용
 - 실제 외부 서비스
 - 공인 IP
 - 클라우드 LB
 - MetalLB

VirtualService에서 추가로 설정하는 것들

- ✓ timeout
- ✓ retries
- ✓ retryOn
- ✓ fault injection
- ✓ route weight
- ✓ header routing

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: resttemplateclient-vs
spec:
  hosts:
    - "*"
  gateways:
    - resttemplate-gateway
  http:
    - match:
        - uri:
            prefix: /books
      timeout: 3s
      retries:
        attempts: 3
        perTryTimeout: 1s
        retryOn: gateway-error,connect-failure,refused-stream
      route:
        - destination:
            host: resttemplateserver.default.svc.cluster.local
            port:
              number: 8080
```


Fault Injection

- ✓ 의도적으로 일부 요청에:
 - 지연(delay)
 - 오류(abort)를 발생
- ✓ 시스템의 회복력(resilience)을 테스트하는 기능

```
fault:  
  delay:  
    percentage:  
      value: 50  
    fixedDelay: 2s
```

50% 요청에 2초 지연

```
fault:  
  abort:  
    percentage:  
      value: 20  
    httpStatus: 500
```

20% 요청에 500 에러
발생

Canary 배포

- ✓ 새 버전(v2)을 전체 배포 전에 일부 사용자에게만 점진적으로 노출하는 배포 방식

```
route:
- destination:
  host: resttemplateserver
  subset: v1
  weight: 90
- destination:
  host: resttemplateserver
  subset: v2
  weight: 10
```

구버전 90%, 신버전 10%
시간이 지날수록 신버전 % 올림

개념

- 목적지 서비스에 대해 어떤 방식으로 트래픽을 처리할지 정의하는 정책
 - ✓ loadBalancer
 - ✓ connectionPool
 - ✓ outlierDetection
 - ✓ TLS
 - ✓ subsets

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: resttemplateserver-dr
spec:
  host:
    resttemplateserver.default.svc.cluster.local
  trafficPolicy:
    loadBalancer:
      simple: ROUND_ROBIN
    connectionPool:
      tcp:
        maxConnections: 100
      http:
        http1MaxPendingRequests: 50
        maxRequestsPerConnection: 10
    outlierDetection:
      consecutive5xxErrors: 3
      interval: 5s
      baseEjectionTime: 30s
```

주요 설정

- ✓ http1MaxPendingRequests: 50
 - 대상 서비스 연결 풀이 꽉 찼을 때 대기열(queue)에 쌓을 수 있는 최대 HTTP 요청 수
 - 현재 처리 가능 연결 초과 ⇒ 대기 큐 ⇒ 최대 50개까지 허용
 - 초과시 circuit breaker 동작 가능
- ✓ maxRequestsPerConnection: 10
 - 하나의 TCP 연결에서 최대 몇 개의 HTTP 요청을 처리한 뒤 연결을 재생성할지
 - 장기 연결 문제 방지
 - 특정 연결 편중 방지

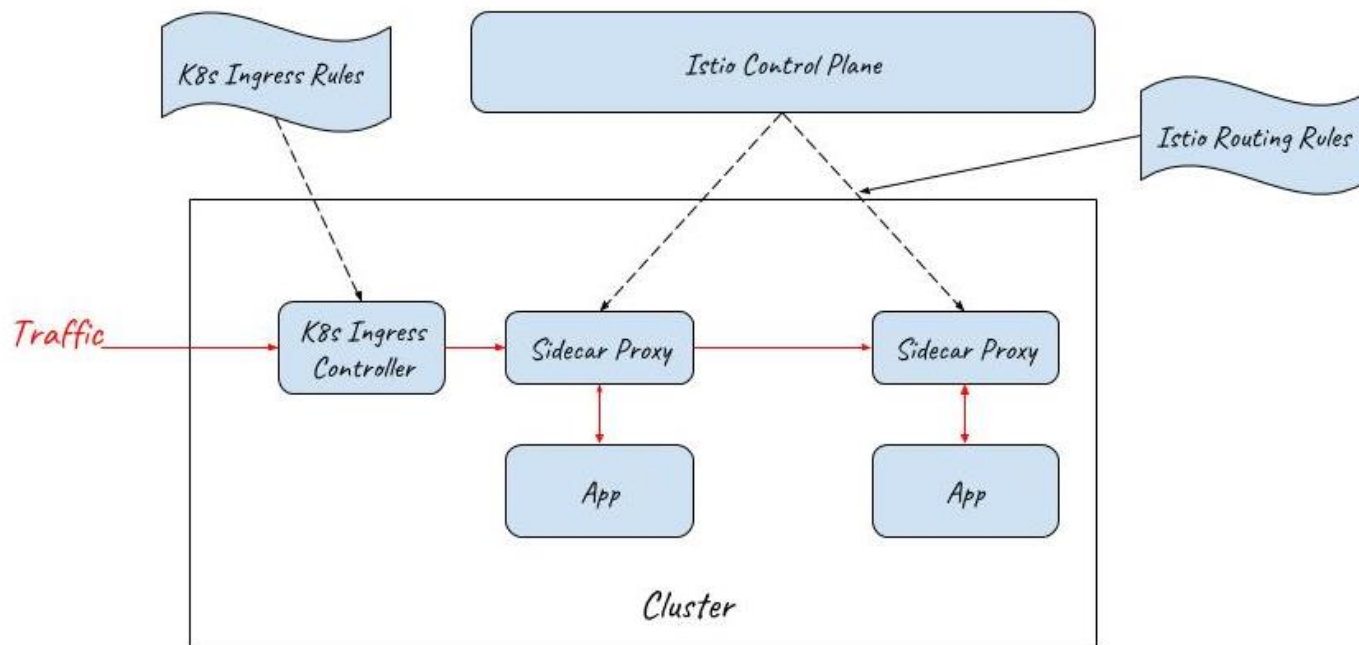
주요 설정

- ✓ Outlier Detection: 비정상(오류율 높은) 인스턴스를 자동으로 트래픽 대상에서 제외(Ejection)하는 기능
- ✓ 사용자 오류 증가, 응답 지연, 장애 확산 막기 위함

설정	의미
consecutive5xxErrors	연속 5xx 에러 허용 횟수
interval	검사 주기
baseEjectionTime	제외 시간
maxEjectionPercent	최대 제외 비율

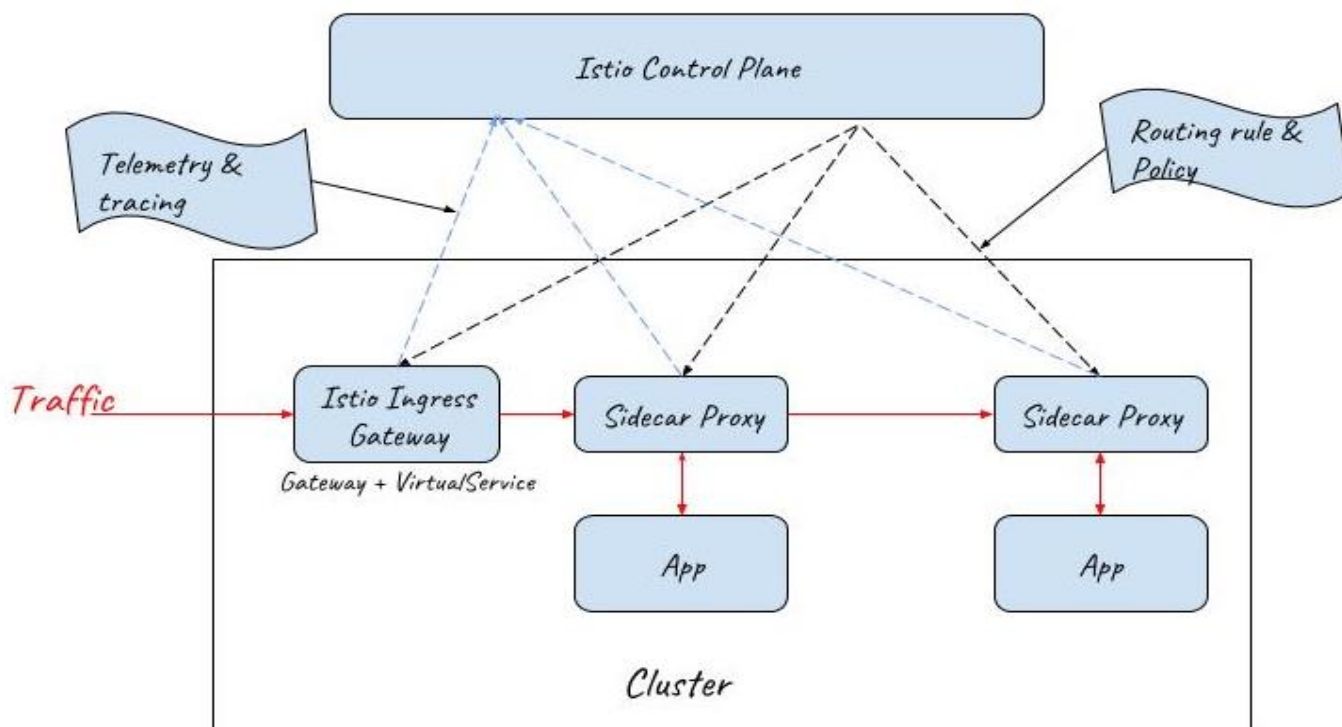
서비스 메시 환경에서 Ingress G/W

- ✓ Ingress는 Istio Control Plane에서 관리할 수 없음
- ✓ Kubernetes Ingress 규칙으로 생성해야 하고, Mesh 와 잘 맞지 않음
- ✓ Ingress는 매우 기본적인 L7 기능만 제공
- ✓ Advanced Routing 규칙, 분산추적, Policy checking, metrics 수집 등 Mesh Sidecar Proxy와 동일한 기능들이 제공되지 않음



Istio G/W

- ✓ Istio Gateway 리소스는 Kubernetes Ingress보다 훨씬 간단함
- ✓ Port, Host, TLS key, 인증 같은 L4-L6 기능만을 구성할 수 있음.
- ✓ Gateway는 메시 내 라우팅 구성에 사용되는 동일한 리소스 인 Istio VirtualService 리소스에 바인딩 됨. 이를 통해 Mesh 내부와 입구에서 위에서 언급한 동일한 기능들을 제공함



Istio G/W에서 추가가 필요한 기능들



K8S Ingress

Load balancing
SSL termination
Virtual hosting



Istio Gateway

Load balancing
SSL termination
Virtual hosting
Advanced traffic routing
Fault injection
Other benefits brought
by Istio

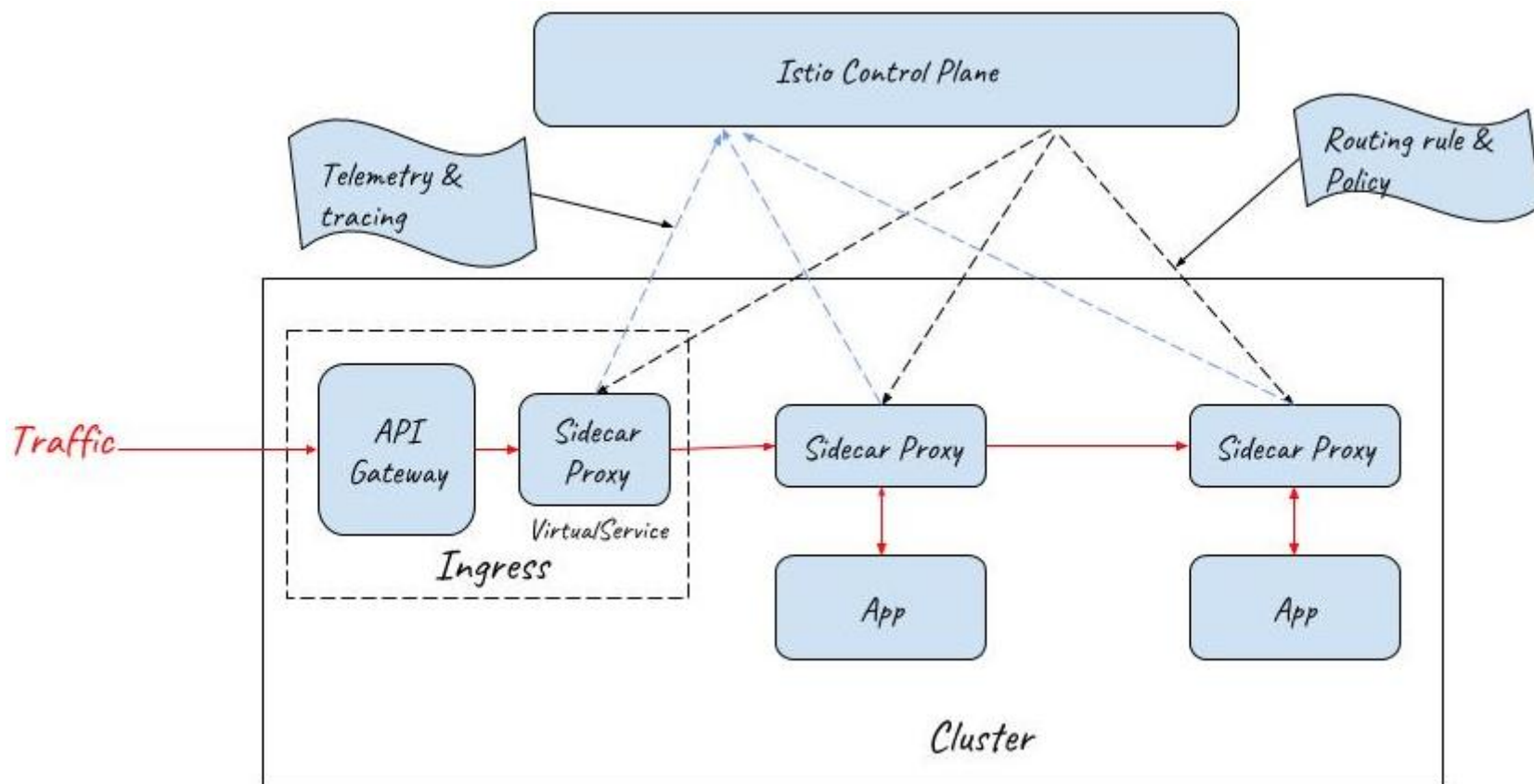


API Gateway

Load balancing
SSL termination
Virtual hosting
Traffic routing
API lifecycle management
API Operation Monitoring
API access authorization
API key management
API transformation
API Billing and Rate limiting

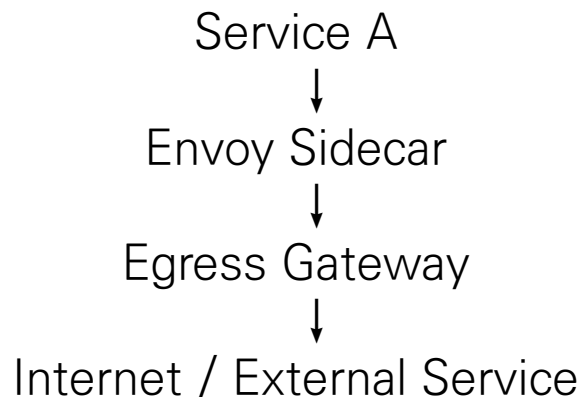
API Gateway + Sidecar Proxy

- ✓ API Gateway와 Sidecar proxy가 Service mesh의 ingress gateway로 사용
- ✓ API Gateway에는 이미 L7 기능이 있으므로 Sidecar proxy는 Istio VirtualService 리소스의 Routing 기능만 제공하면 됨

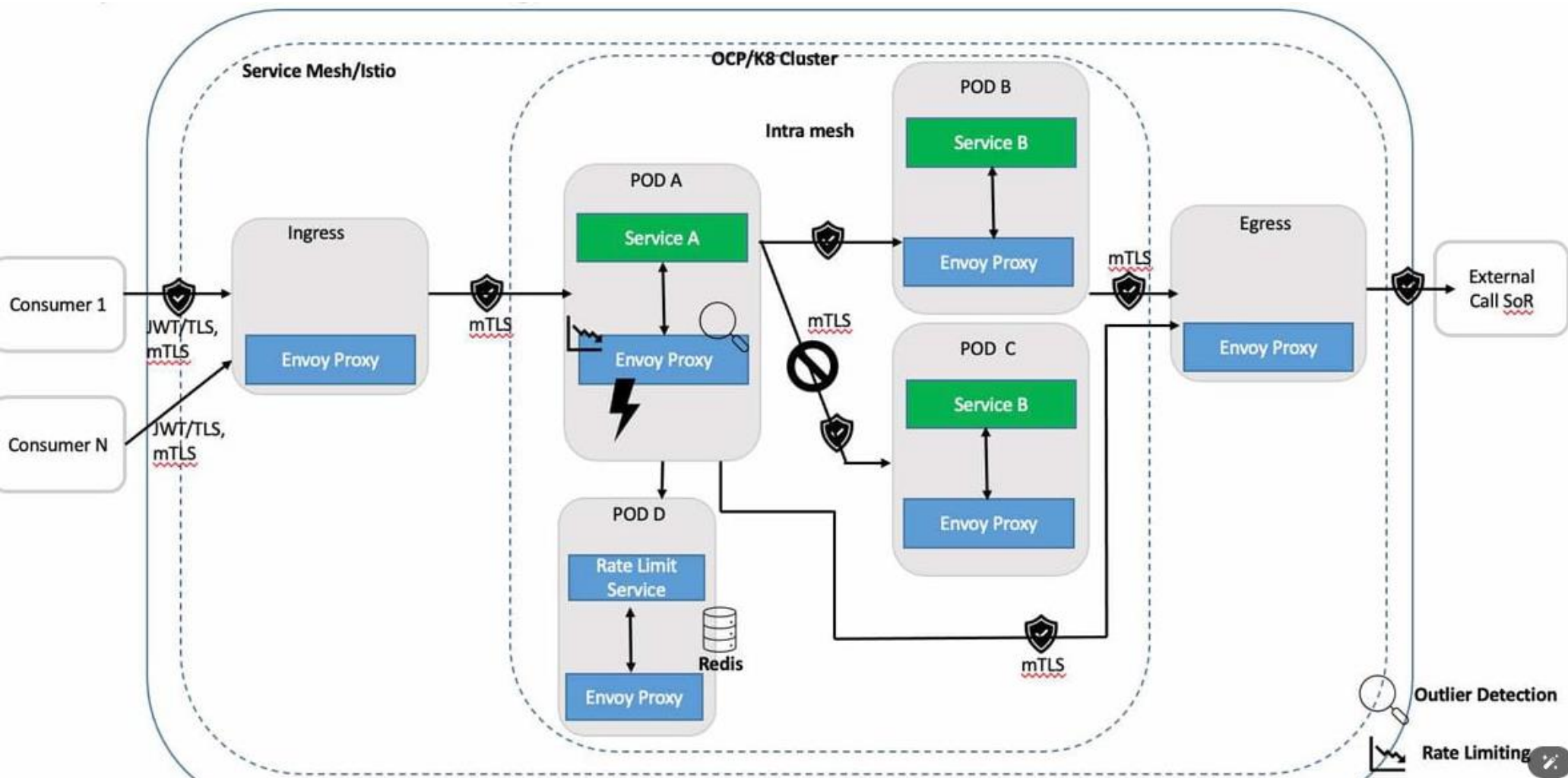


목적

- ✓ Egress는 메시 내부 서비스가 외부 시스템(클러스터 밖)으로 나갈 때의 트래픽 제어
 - 외부 접속 통제
 - 허용된 도메인만 접근
 - TLS 정책감사 로그
 - 보안, 트래픽 모니터링
- ✓ 내부 서비스가 아래 호출하는 상황
 - Stripe API 호출
 - 외부 MySQL 접속
 - Kafka SaaS
 - Google API
 - 외부 REST API



개념도



활용 방법

- ✓ 외부 접근 제한 명시하기 위해 ServiceEntry 사용
 - DNS 등록, TLS 정책, Egress Gateway 연계, 외부 서비스 카탈로그화
 - meshConfig.outboundTrafficPolicy.mode
 - ⇒ ALLOW: 등록 안 된 외부 주소도 허용
 - ⇒ REGISTRY_ONLY: ServiceEntry 등록 대상만 허용

기능	효과
보안	무분별한 외부 호출 방지
감사	모든 outbound 추적
정책	특정 SaaS만 허용
NAT 통제	고정 출구 IP
Zero Trust	강화